Self-Balancing and AVL Trees

Overview

- Tree Balance
- AVL Tree
- Rotations
- Using AVL trees

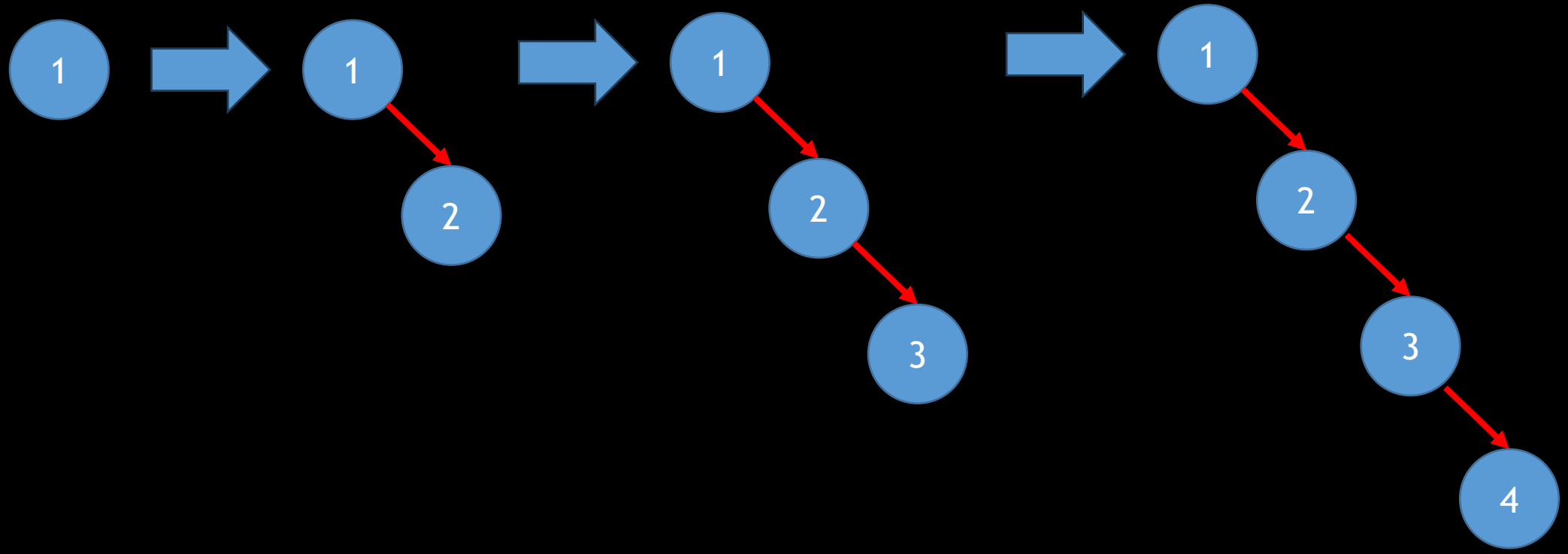
Binary Search Trees - Performance

- Depends on their balance
  - Balanced $O(\log_2 n)$
  - Unbalanced $O(n)$

- Complexity arises from the need to maintain order during insertion/deletion operations

Degenerate Trees

- Insert values sequentially
  - E.g. 1, 2, 3, 4



Degenerate Trees

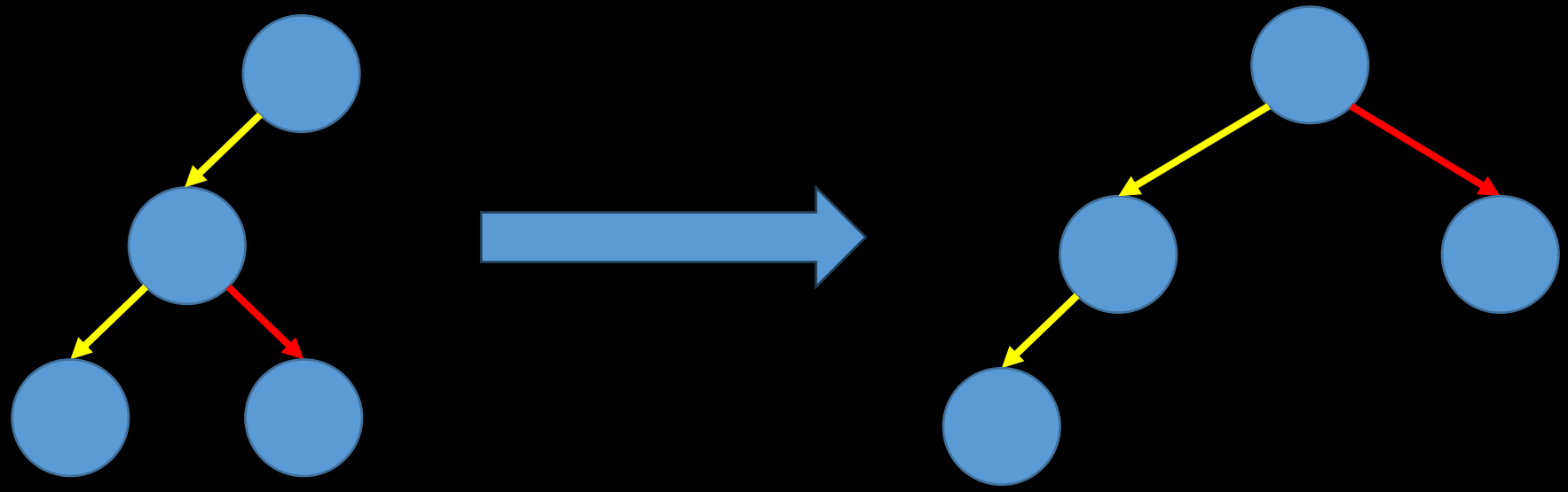
- Search complexity becomes O(n)
  - No better than a linked list
- Theoretical best search complexity is $O(\log_2 n)$
- Balanced binary tree
  - Maximum depth of each side of each node must be no more than one in difference
- A lot of insertions/deletions can unbalance a binary tree

Binary Search Tree - Balance



Binary Search Trees – Self-Balancing

- Keeps the height of the tree minimal
- Balance is checked during insert/delete operations
- Actions are taken when imbalance is detected
  - These actions (rotations) restore balance



AVL Tree

- Adelson-Velsky & Landis
  - Адельсóн-Вéльский & Лáндис

- Insertion and Deletion methods attempt to maintain the tree’s balance

- Tradeoff
  - Methods become more expensive
  - Searching becomes faster

Balance Factor

- Each AVL tree node has a height (depth)
- Height is used to calculate the Balance Factor of its subtrees
  - Balance Factor = LeftHeight – RightHeight

- Height: Node height takes the largest subtree height and adds 1
  - Leaf node height = 0
  - Null children have height -1

- Balance Factor: Signed difference between the height of the right and left subtree
  - Balance Factor 0: Left and right subtrees are the same depth
  - Balance Factor 1: Left subtree is one level deeper
  - Balance Factor -1: Right subtree is one level deeper
  - abs(Balance Factor) > 1: Tree is unbalanced

Balance Factor

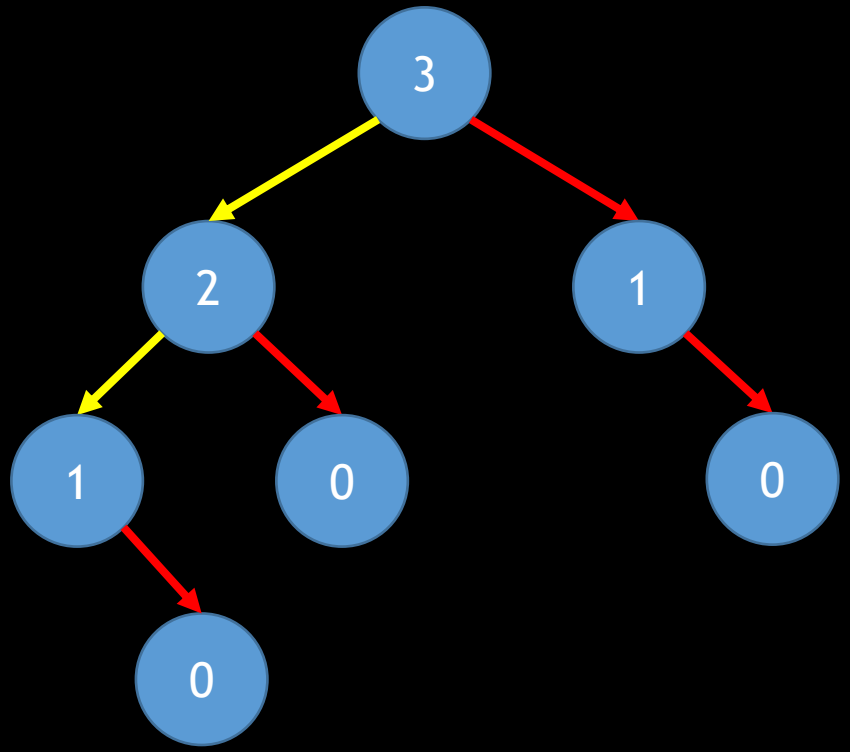
- How to find the height of a binary tree?

```
findHeight ( rootNode )
        if rootNode == null
                return 0
        else
                leftHeight = findHeight (rootNode.left)
                rightHeight = findHeight (rootNode.right)

                return max(leftHeight, rightHeight) + 1
```

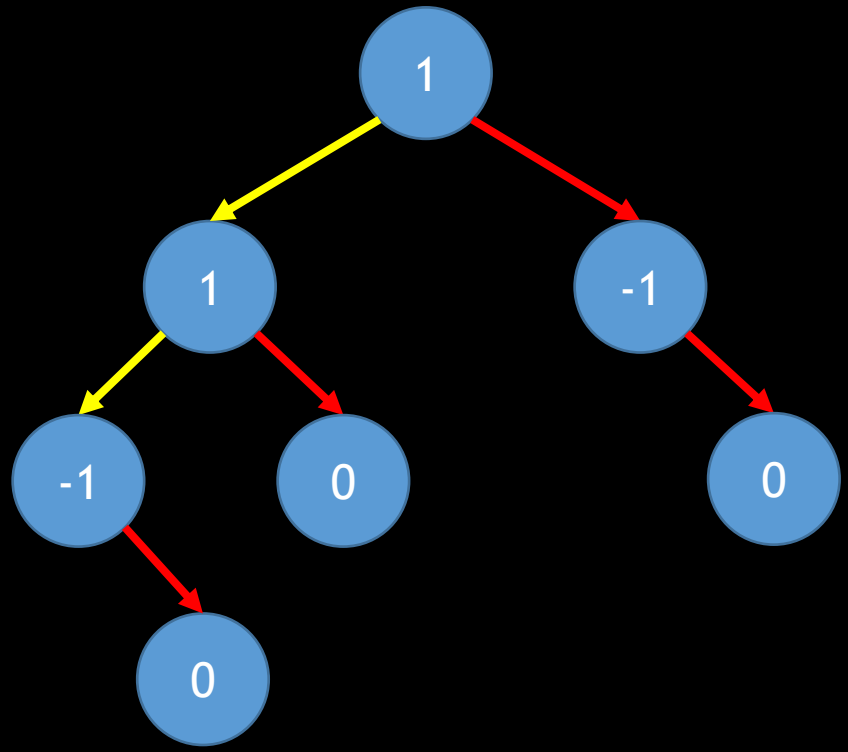
Height vs Balance Factor

Height



Balance Factor

- BF = LeftHeight – RightHeight



- Virtual child = -1

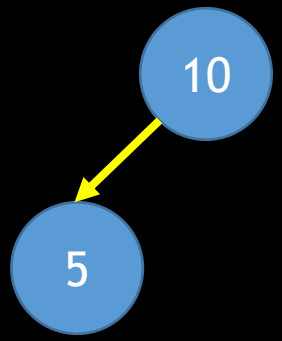
Insertion

- Insert as per a normal BST
- Starting at this new leaf node work back up towards the root
  - Calculate the new Balance Factor for each node
  - Any node where the abs(Balance Factor) > 1 need to be rebalanced
  - Stop once the root of the tree is reached

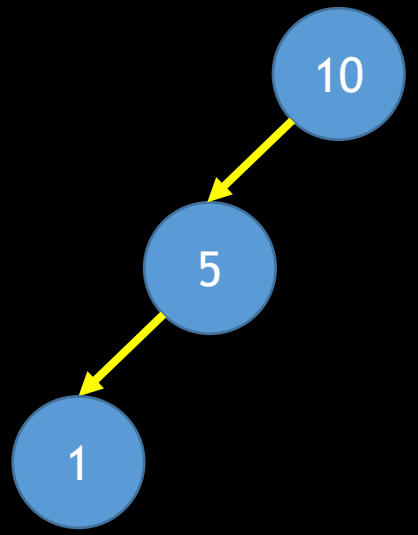
- How do we rebalance?
  - Rotations!

Rotation

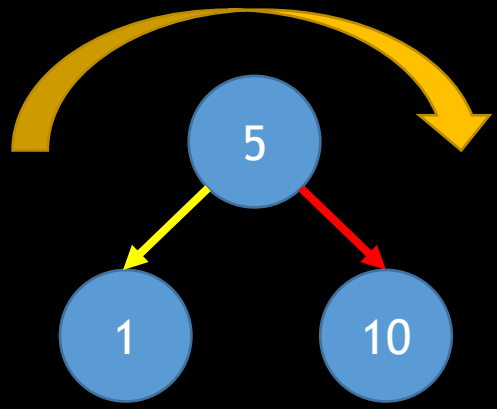
- Initial Tree



After BST Insertion

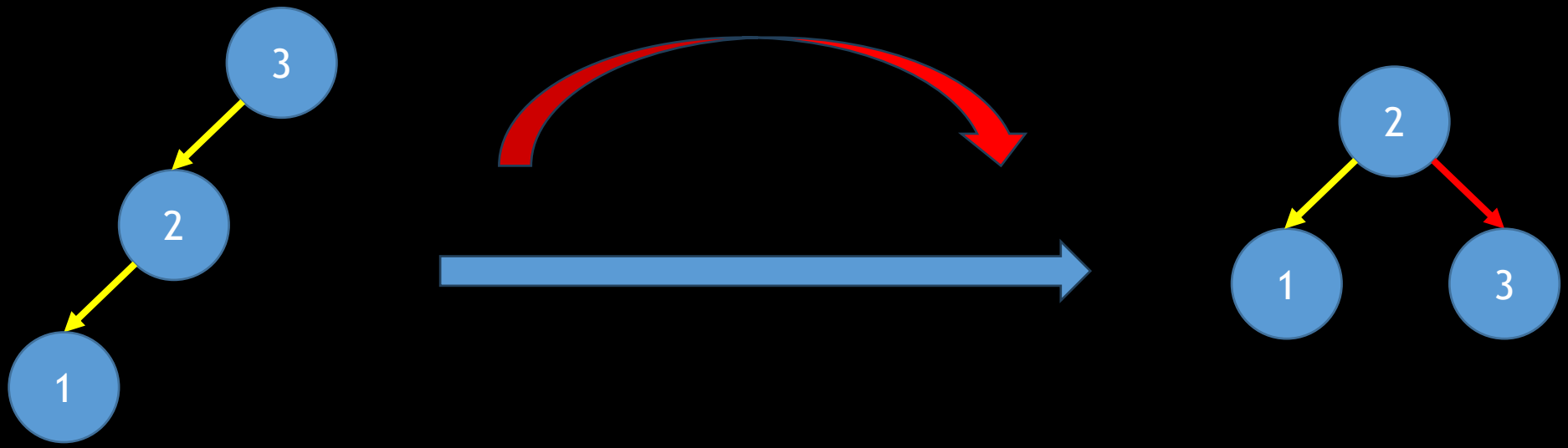


Rotation to Balance



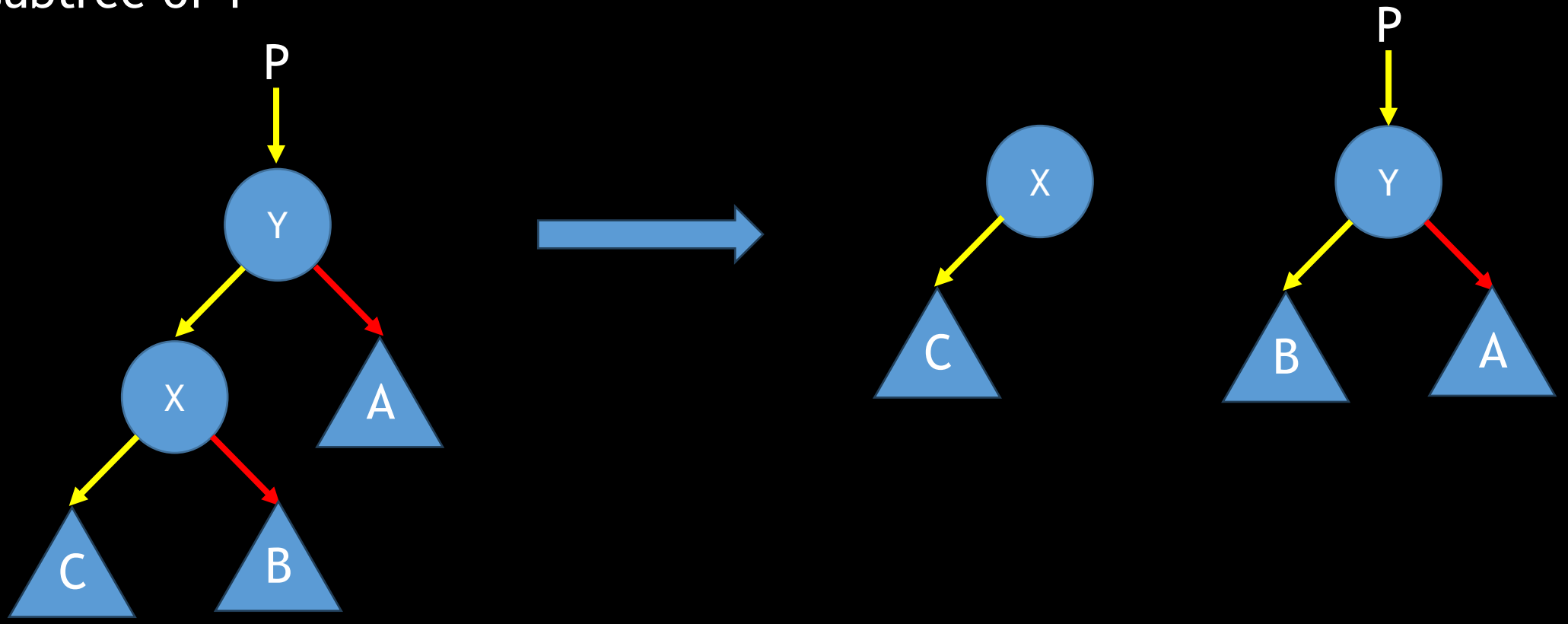
Rotation - Right Rotation

- Right rotation



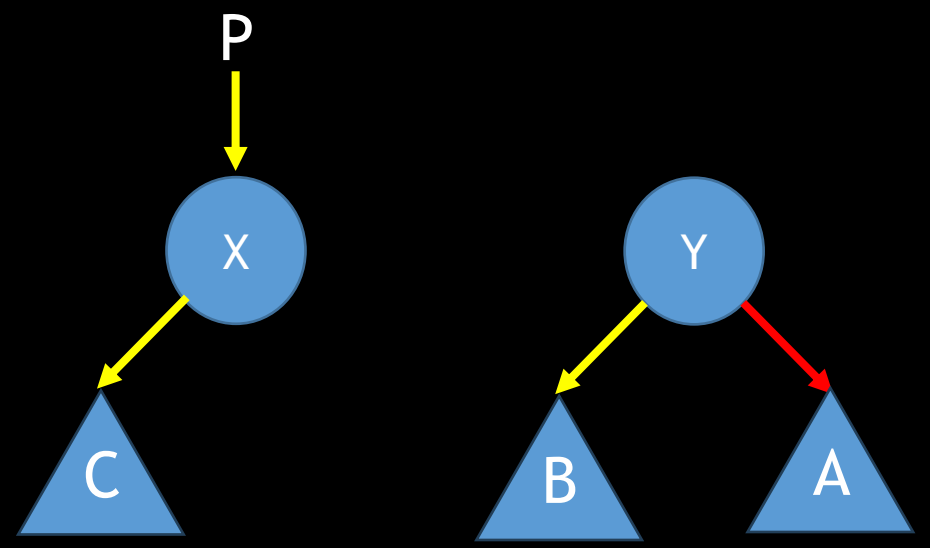
Rotation - Right Rotation

- IF X has a right subtree (B), make B the left subtree of Y



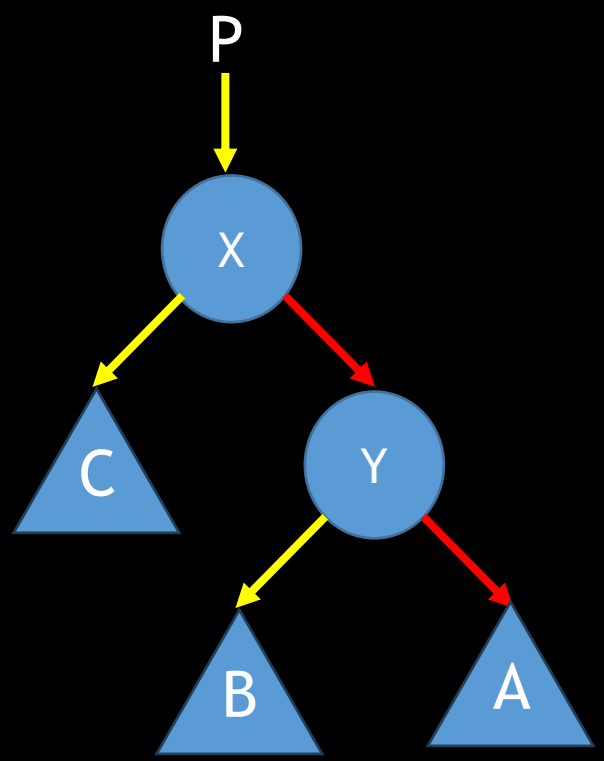
Rotation - Right Rotation

- IF parents of Y is null, make X the root of the tree
- ELSE IF Y is the right child of its parent P, make X the right child of P
- ELSE assign X as the left child of P



Rotation - Right Rotation

- Make X the parent of Y



Rotation - Right Rotation

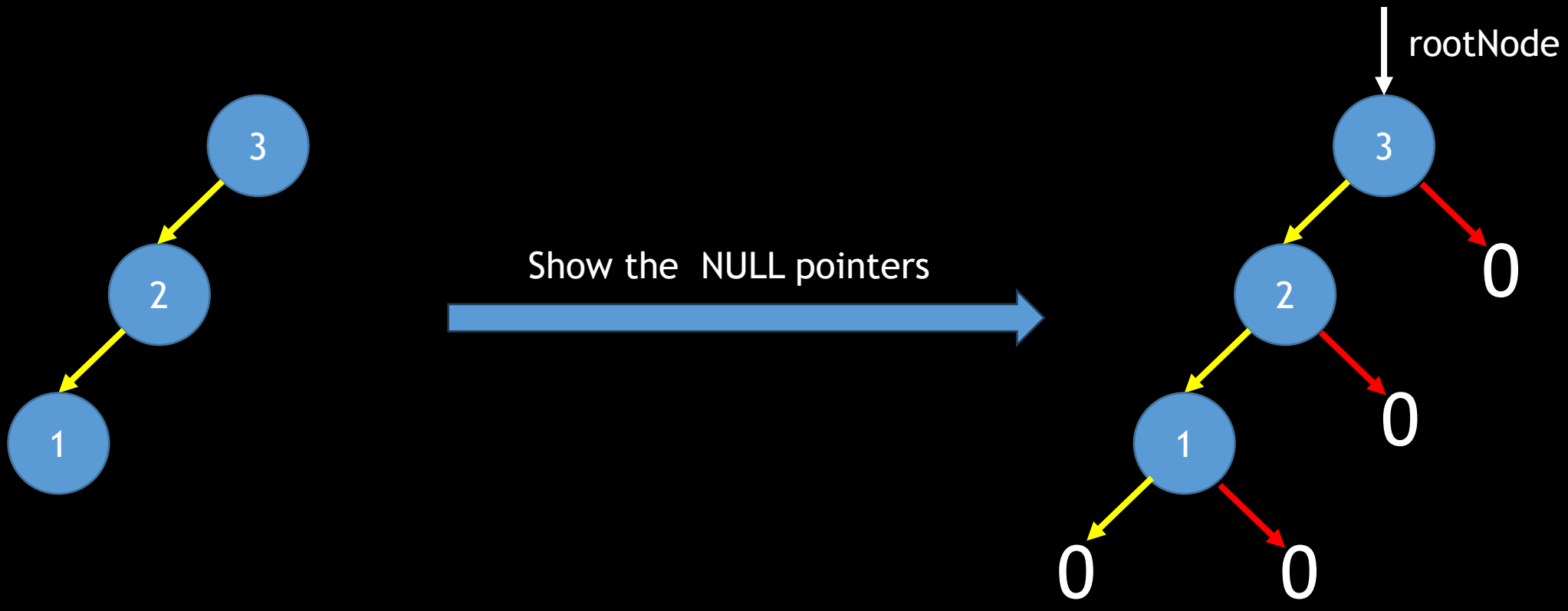
- Right rotate algorithm

```
rightRotate (rootNode )
        leftChild = rootNode.left
        temp = leftChild.right

        rootNode.left = temp
        leftChild.right = rootNode

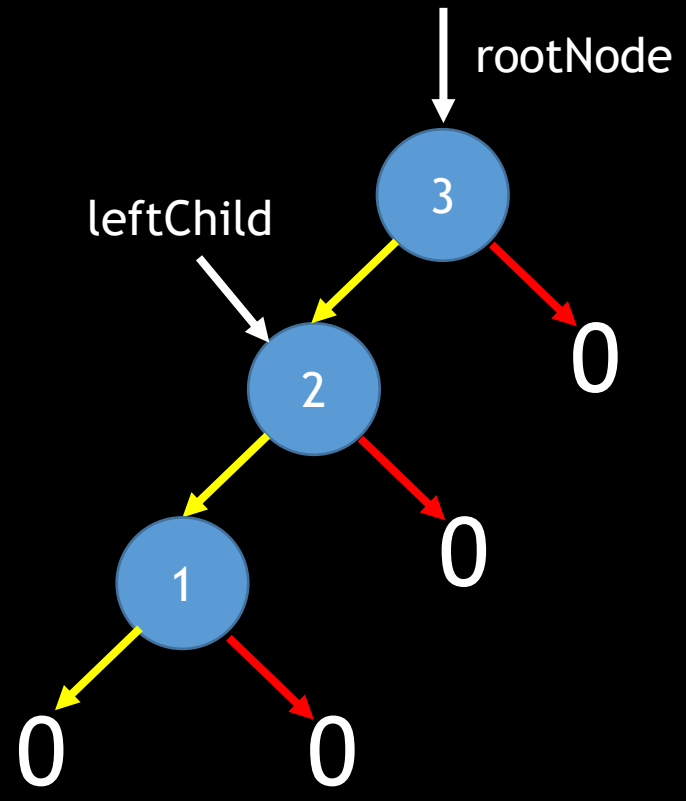
        return leftChild
```

Rotation - Right Rotation



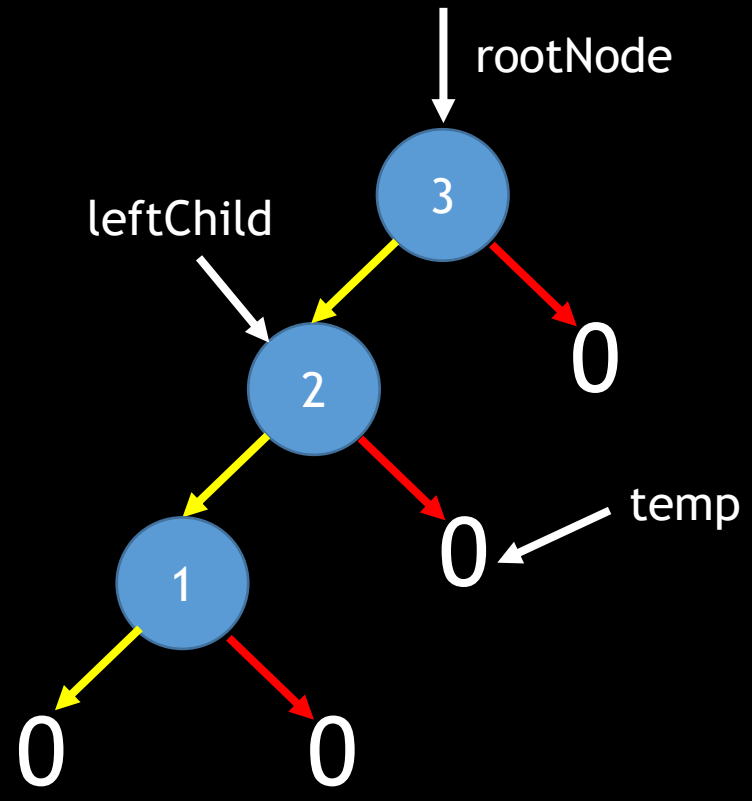
Rotation - Right Rotation

leftChild = rootNode.left



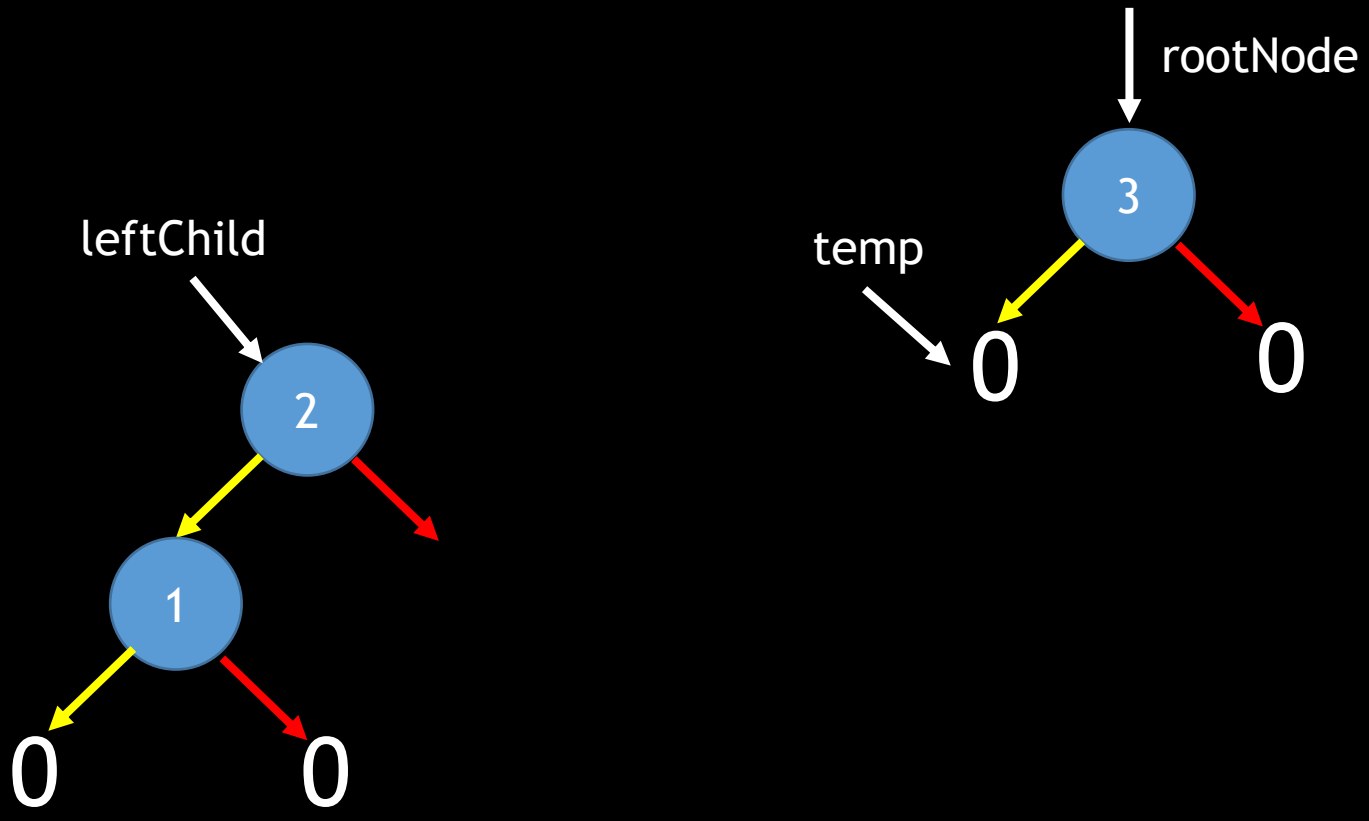
Rotation - Right Rotation

temp = leftChild.right



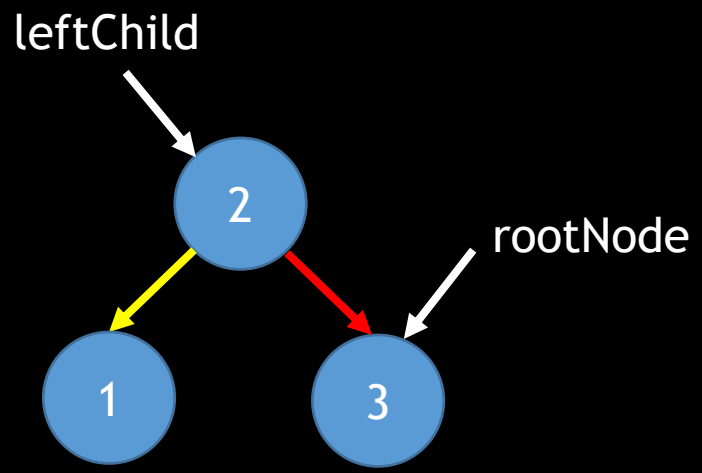
Rotation - Right Rotation

rootNode.left = temp



Rotation - Right Rotation

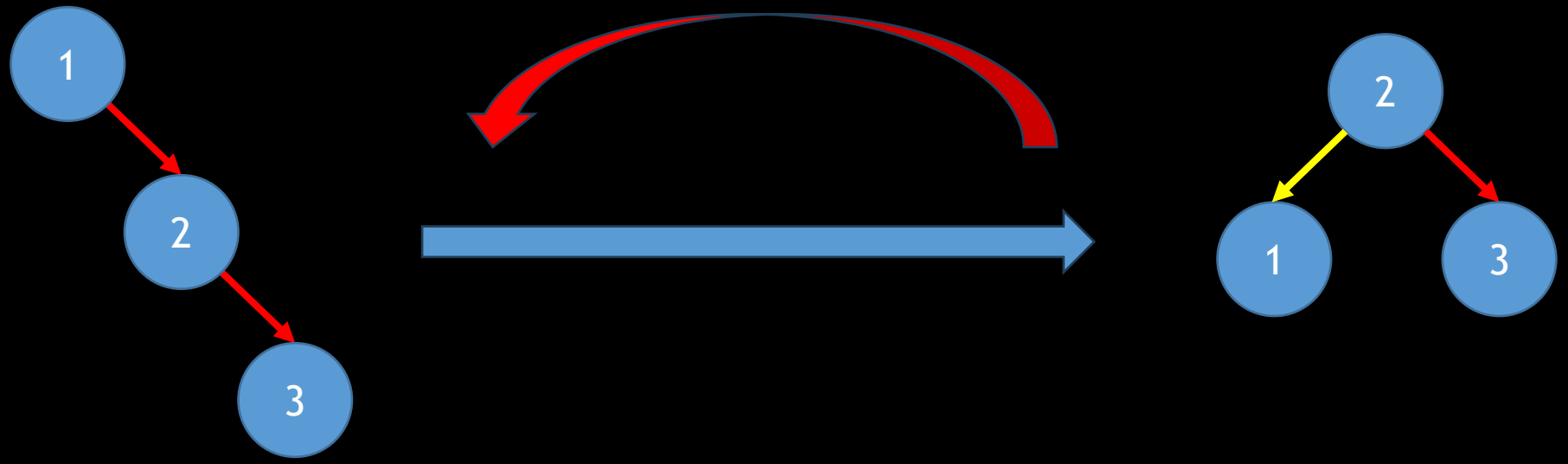
leftChild.right = rootNode



return leftChild

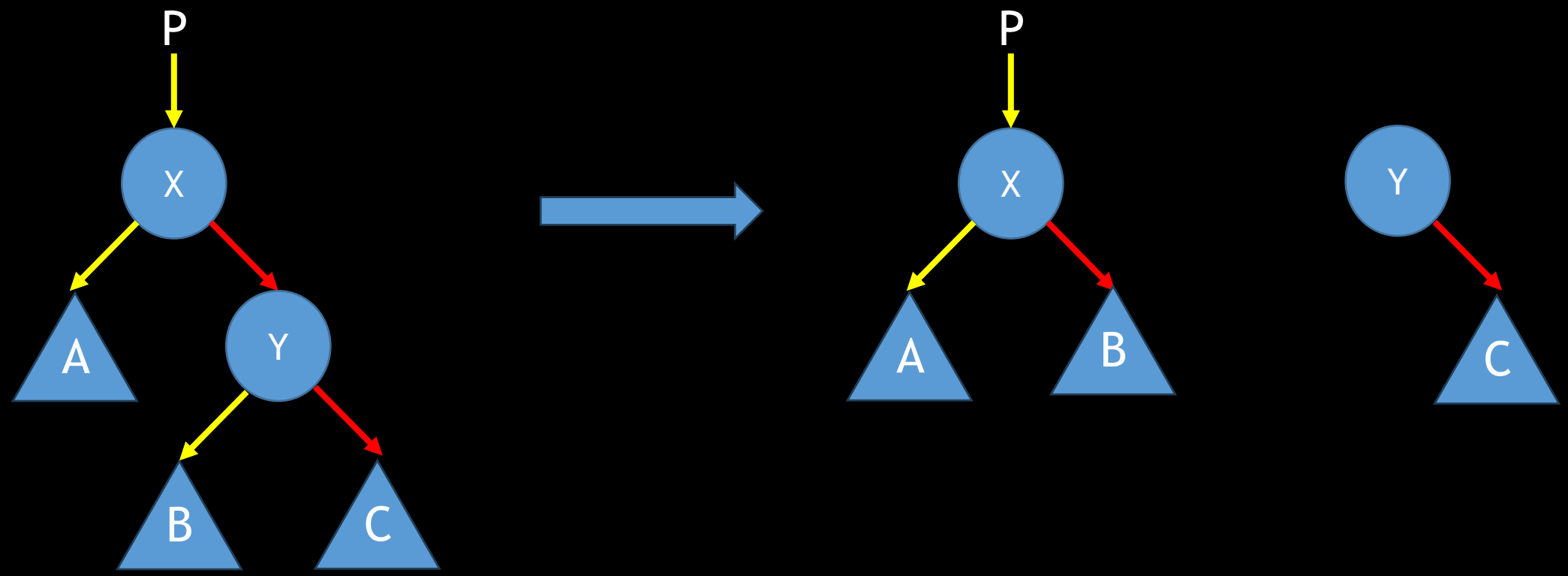
Rotation - Left Rotation

- Left rotation



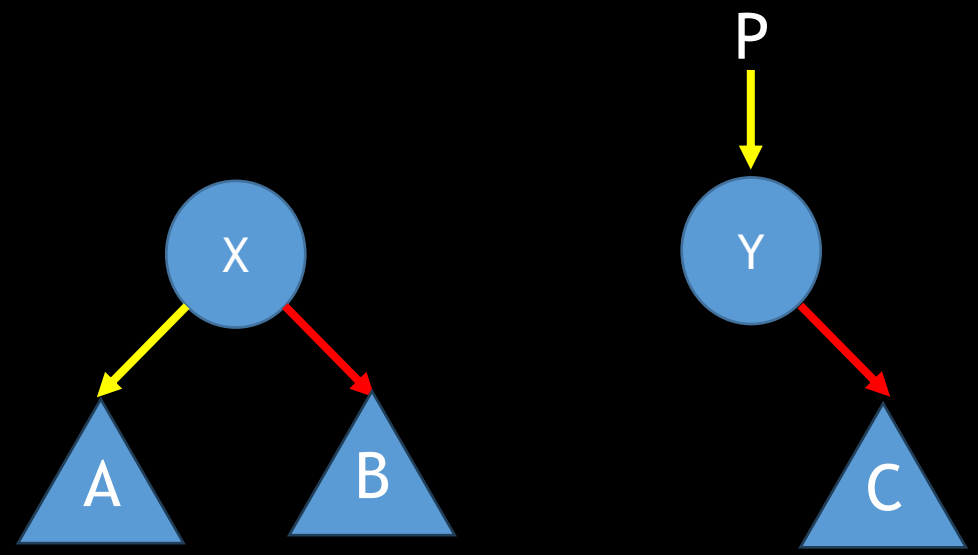
Rotation - Left Rotation

- IF Y has a left subtree (B), assign B as the right subtree of X



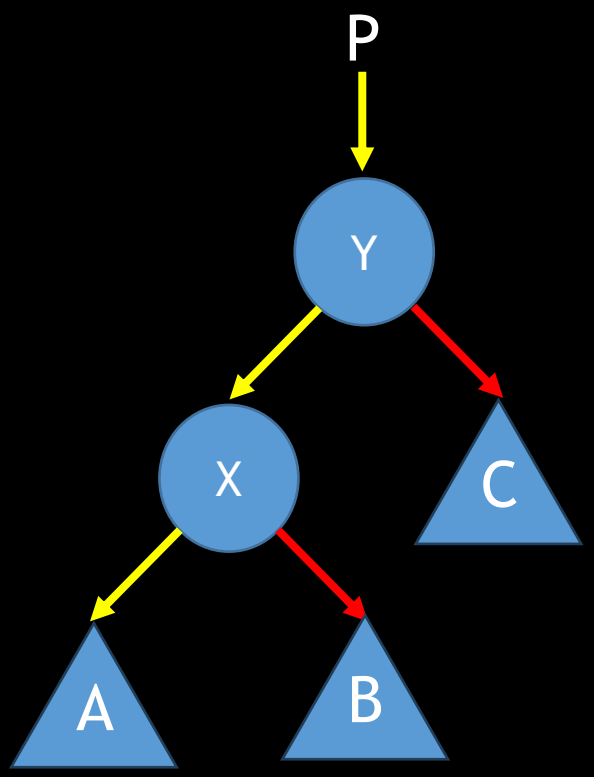
Rotation - Left Rotation

- IF P of X is null, make Y the root of the tree
- ELSE IF X is the left child of P, make Y the left child of P
- ELSE make Y the right child of P



Rotation - Left Rotation

- Make Y the parent of X



Rotation - Left Rotation

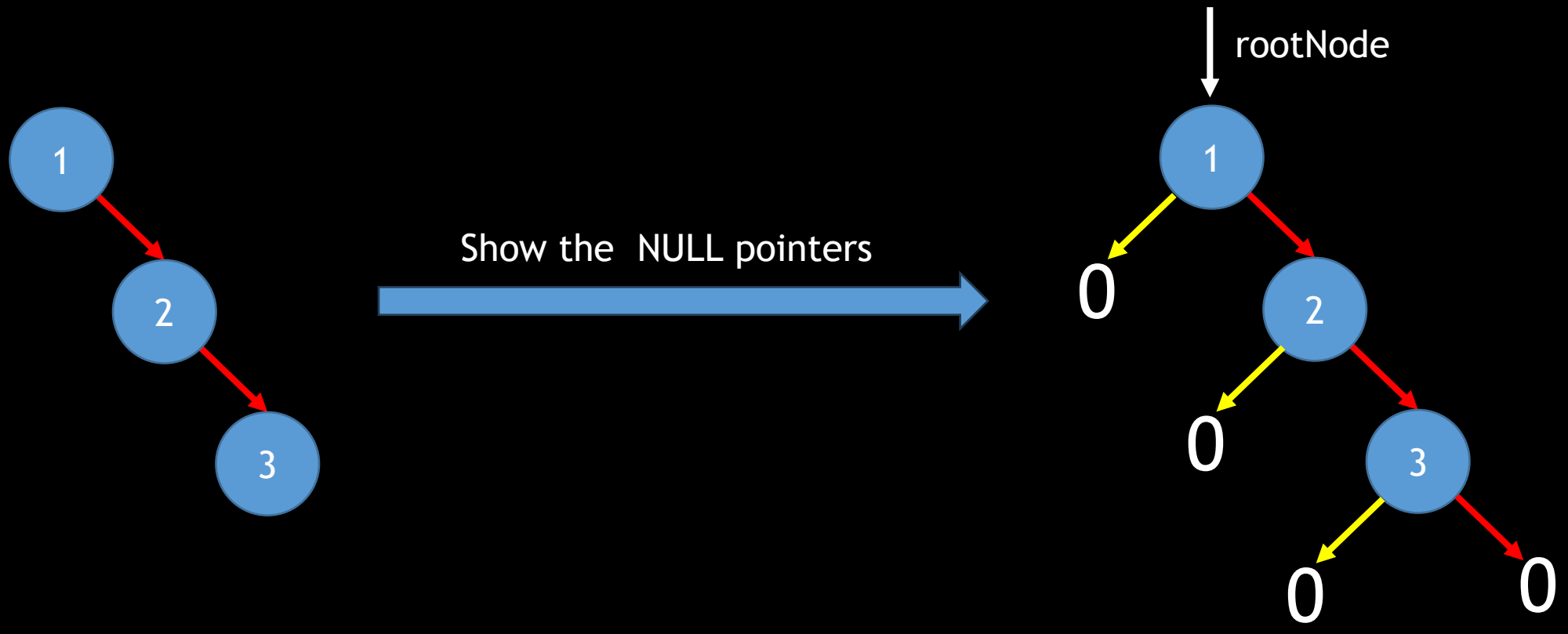
- Left rotate algorithm

```
leftRotate ( rootNode )
        rightChild = rootNode.right
        temp = rightChild.left

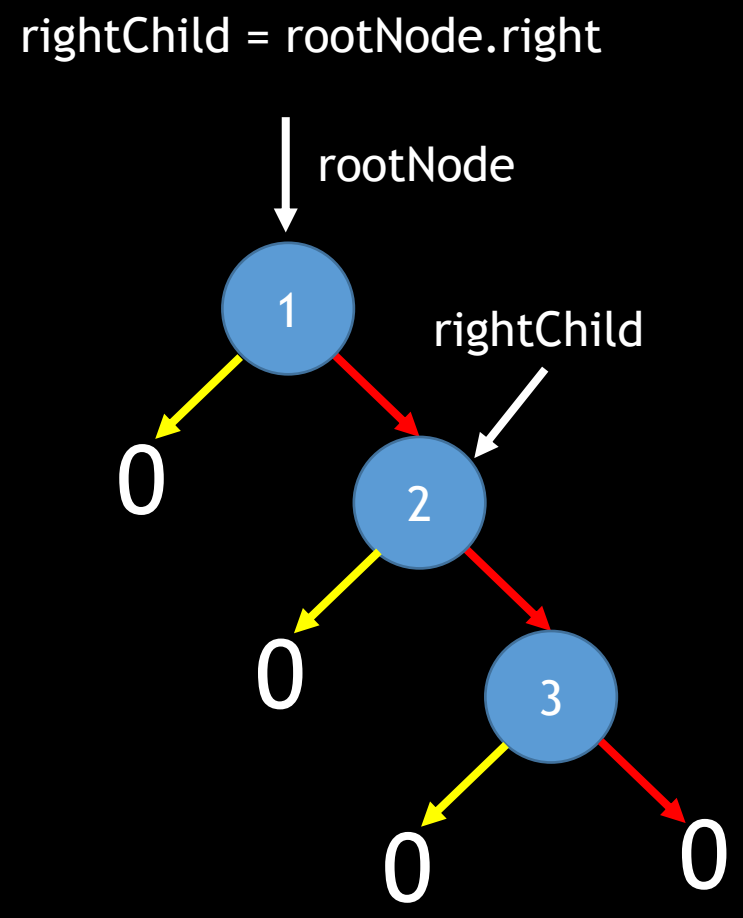
        rootNode.right = temp
        rightChild.left = rootNode

        return rightChild
```

Rotation - Left Rotation

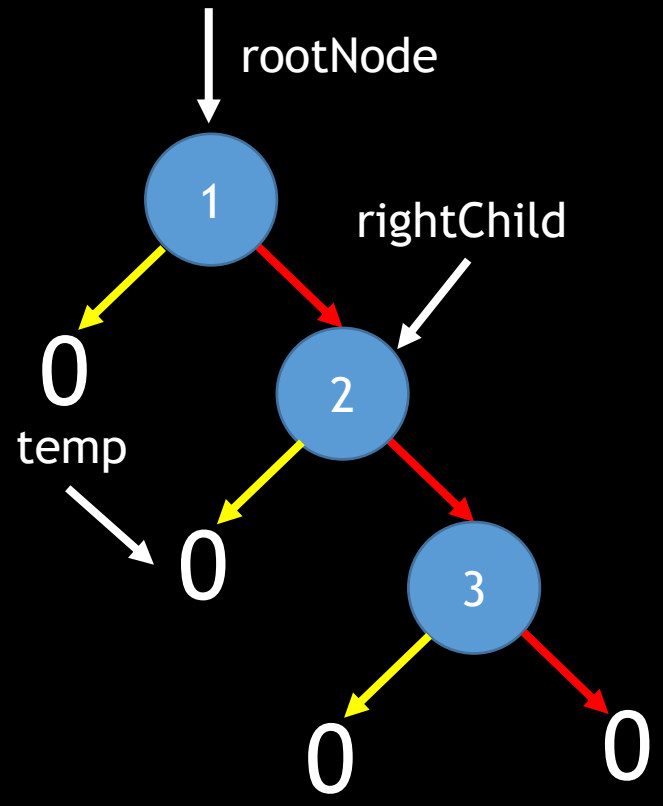


Rotation - Left Rotation



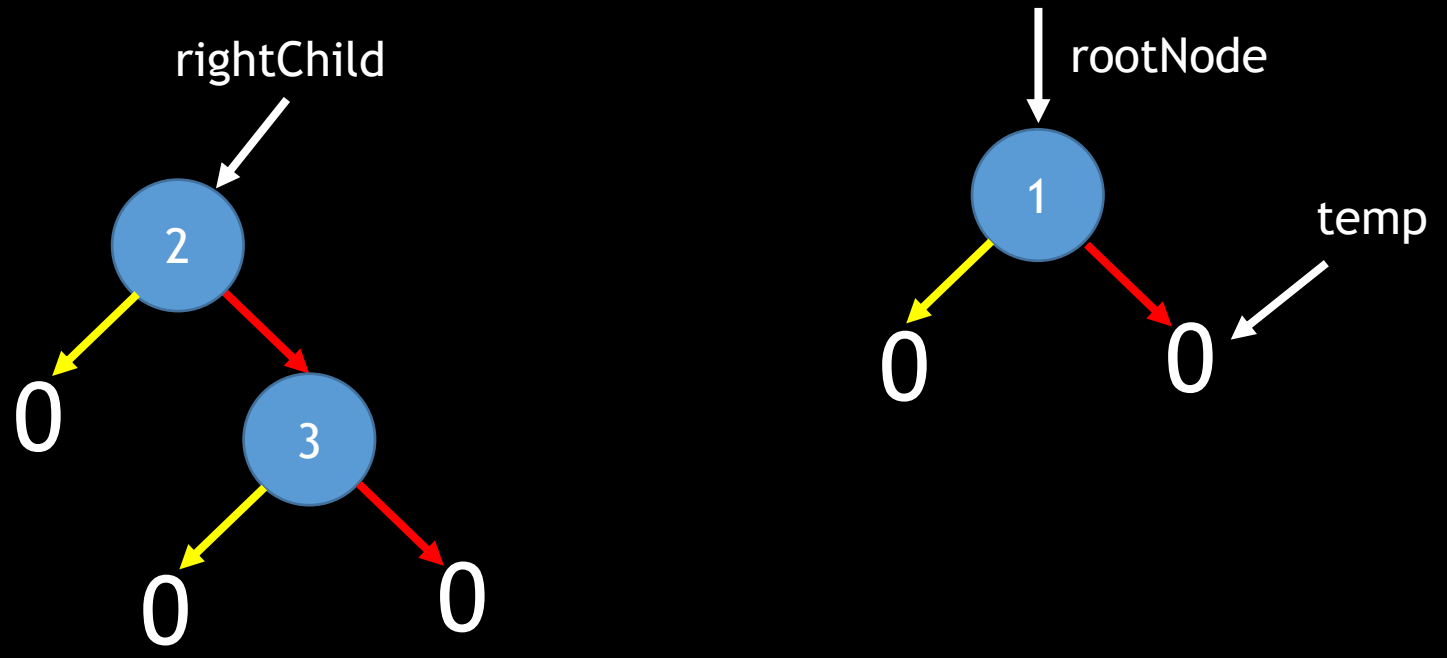
Rotation - Left Rotation

temp = rightChild.left



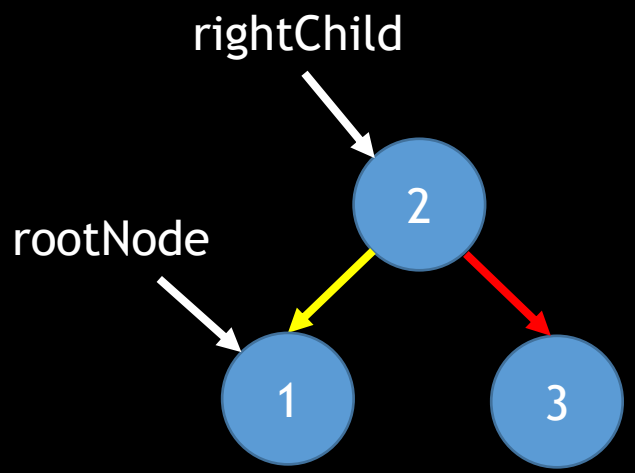
Rotation - Left Rotation

rootNode.right = temp



Rotation - Left Rotation

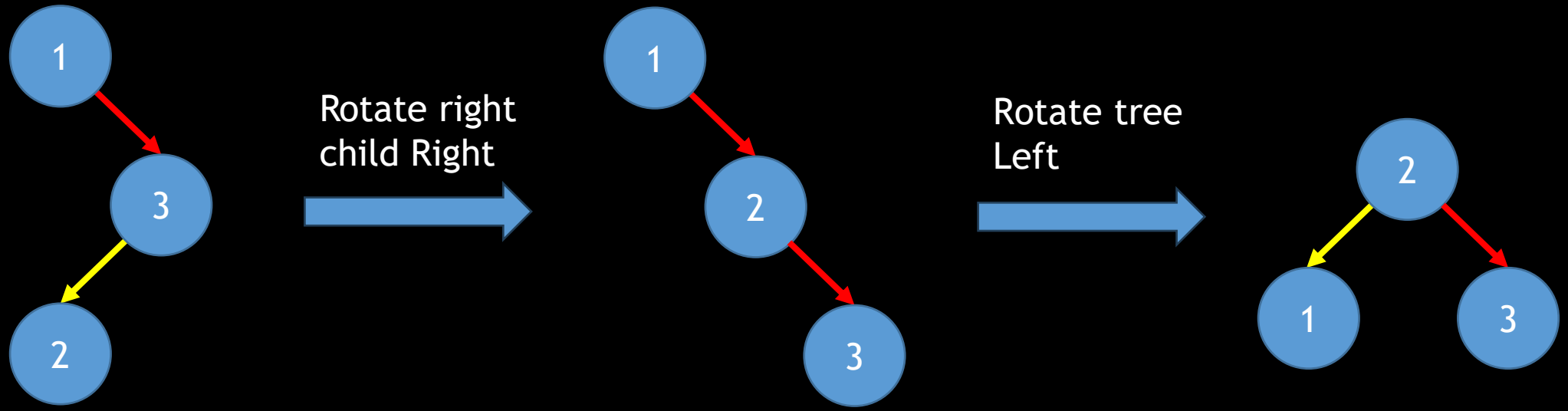
rightChild.left = rootNode



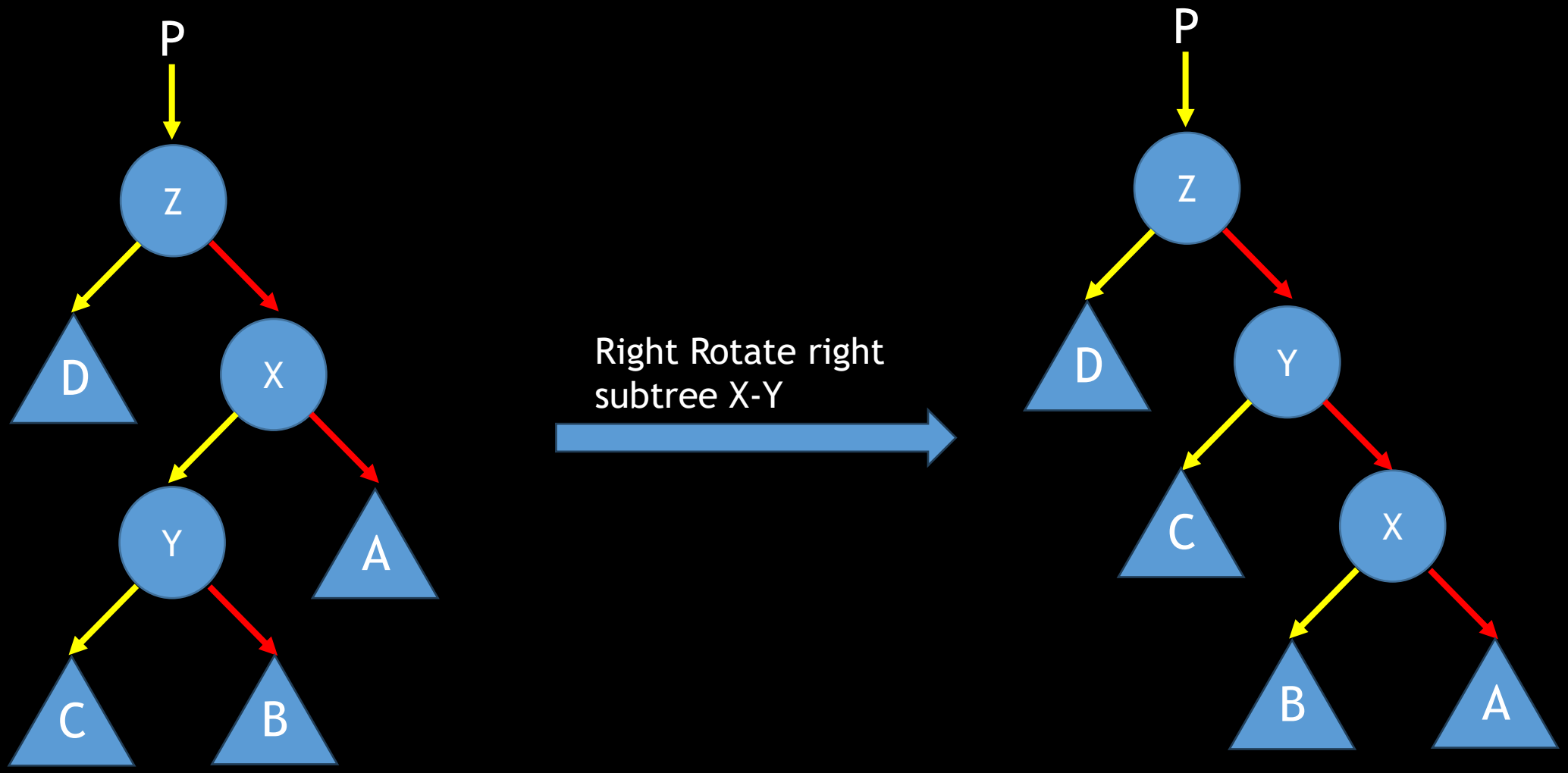
return rightChild

Rotations – Right-Left Rotation

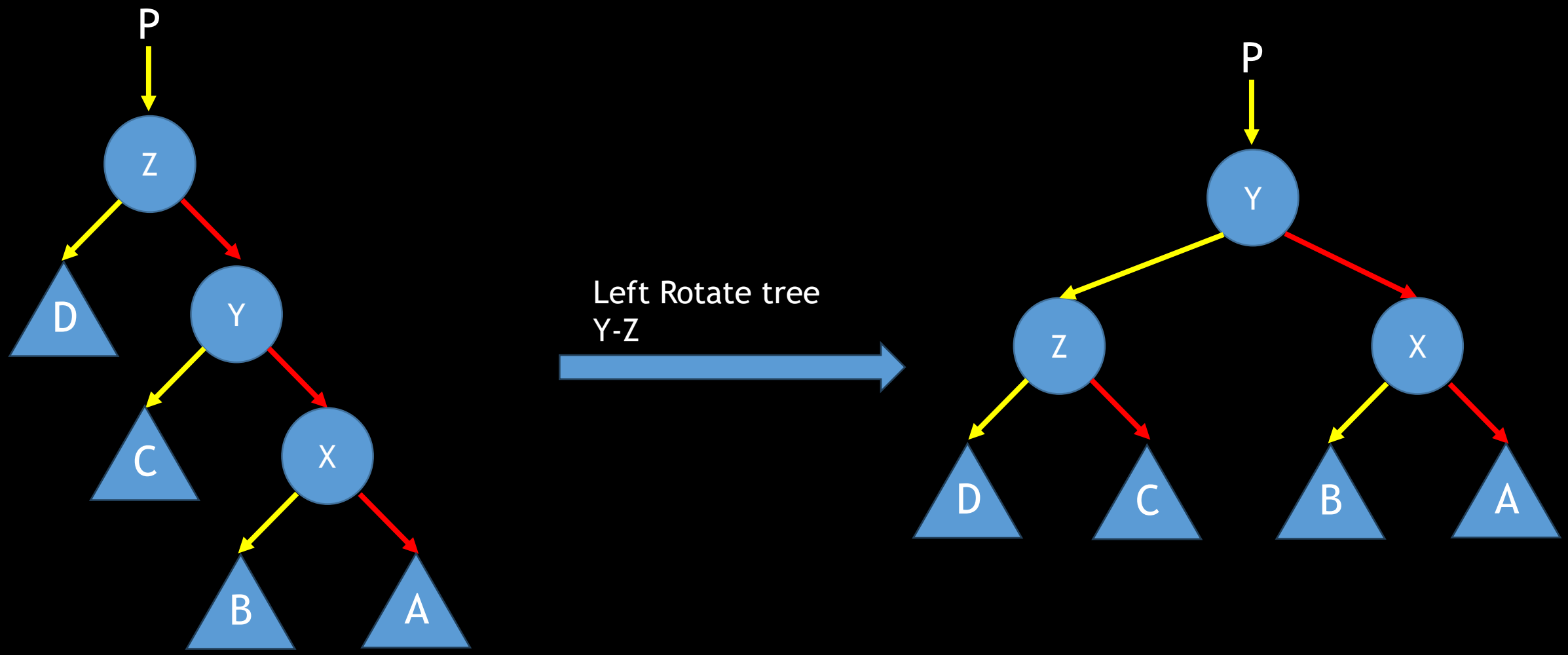
- Right-Left rotation



Rotations – Right-Left Rotation

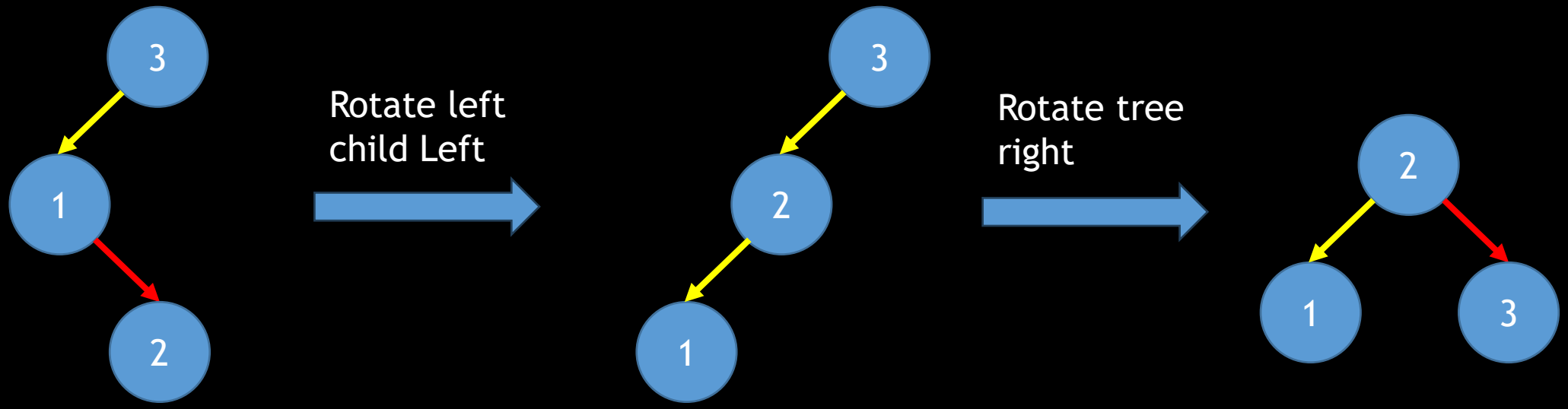


Rotations – Right-Left Rotation

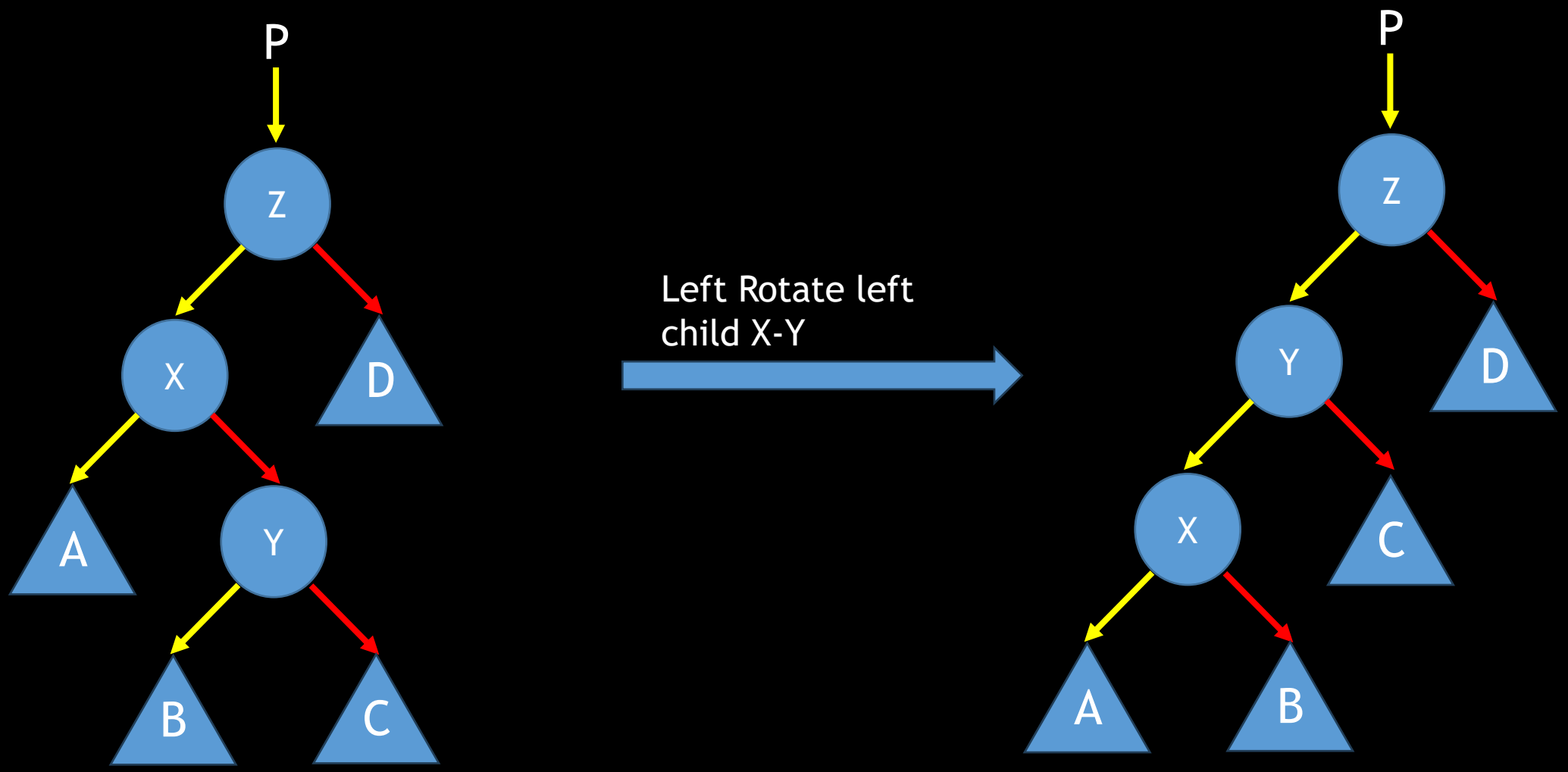


Rotations – Left-Right Rotation

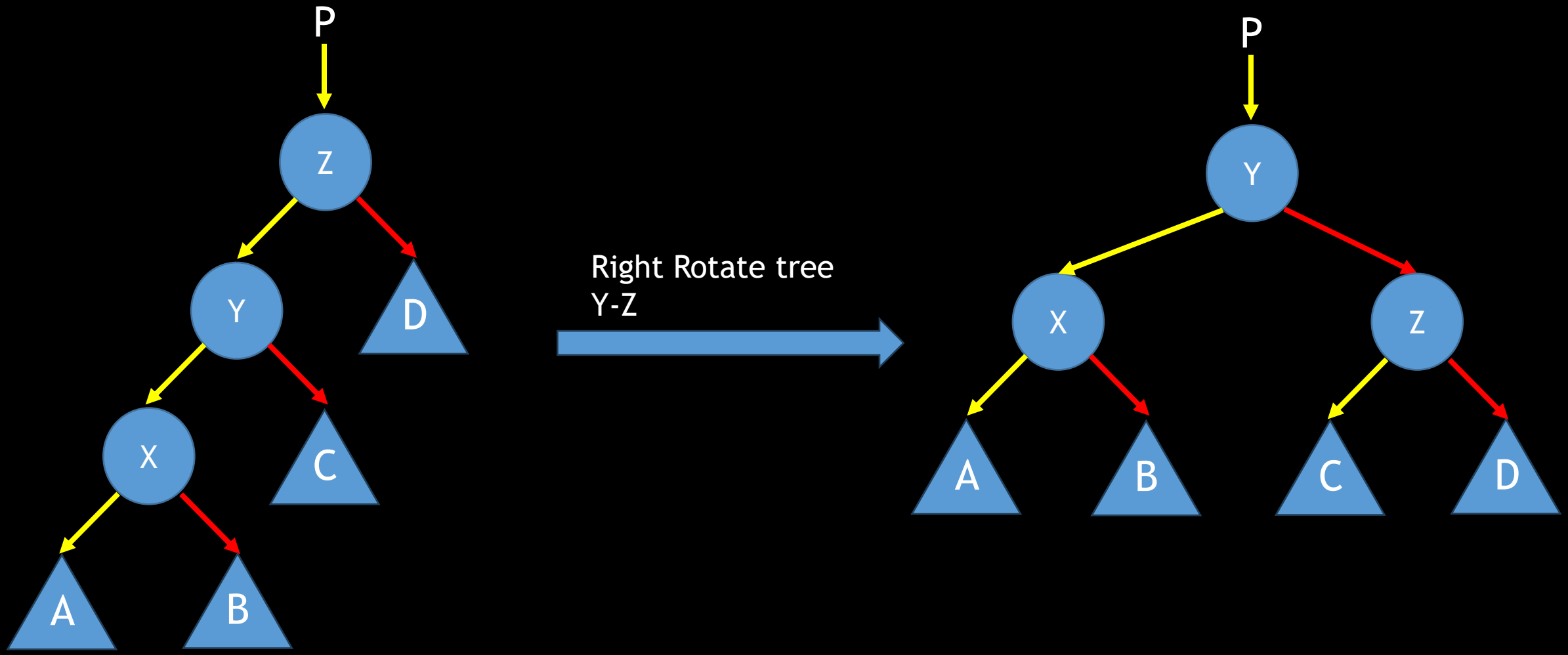
- Left-Right rotation



Rotations – Left-Right Rotation



Rotations – Left-Right Rotation



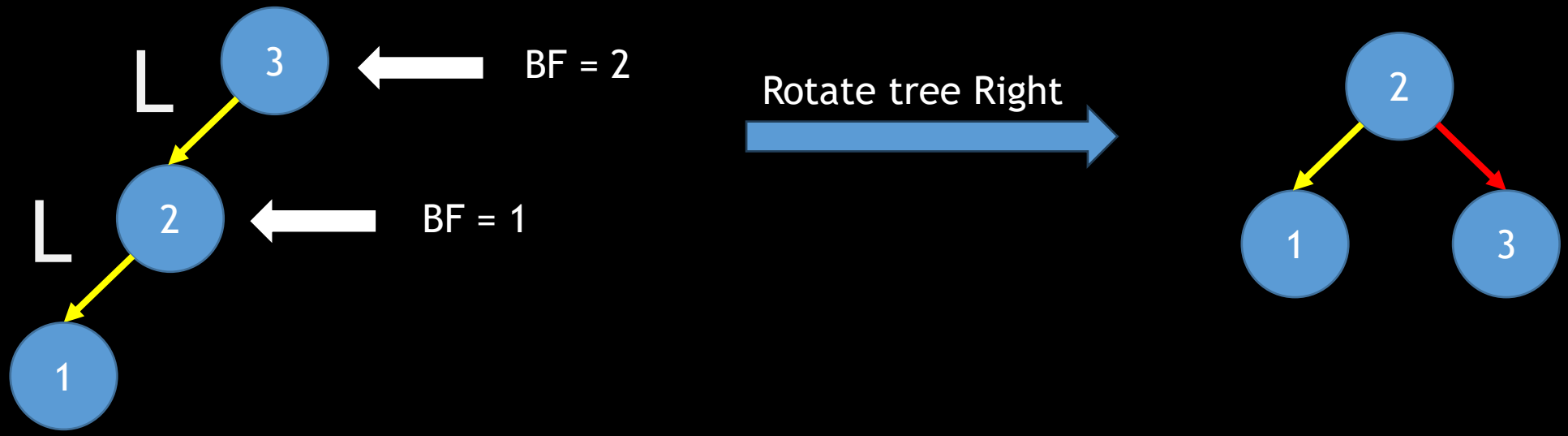
Rotations

- Rotations are determined starting from the unbalanced node
- Determine which child has the most height (Left or Right) to get the first direction
- Repeat check on that child to get the second direction
  - A node with children of equal height can use either
  - Balance Factor = LeftHeight – RightHeight

Depth in Children		Balance(node)	Balance(child)	Rotations	
Left Left	LL	node > 1	node(left) >= 0	Right	(Single)
Right Right	RR	node < -1	node(right) <= 0	Left	(Single)
Left Right	LR	node > 1	node(left) < 0	Left Right	(Double)
Right Left	RL	node < -1	node(right) > 0	Right Left	(Double)

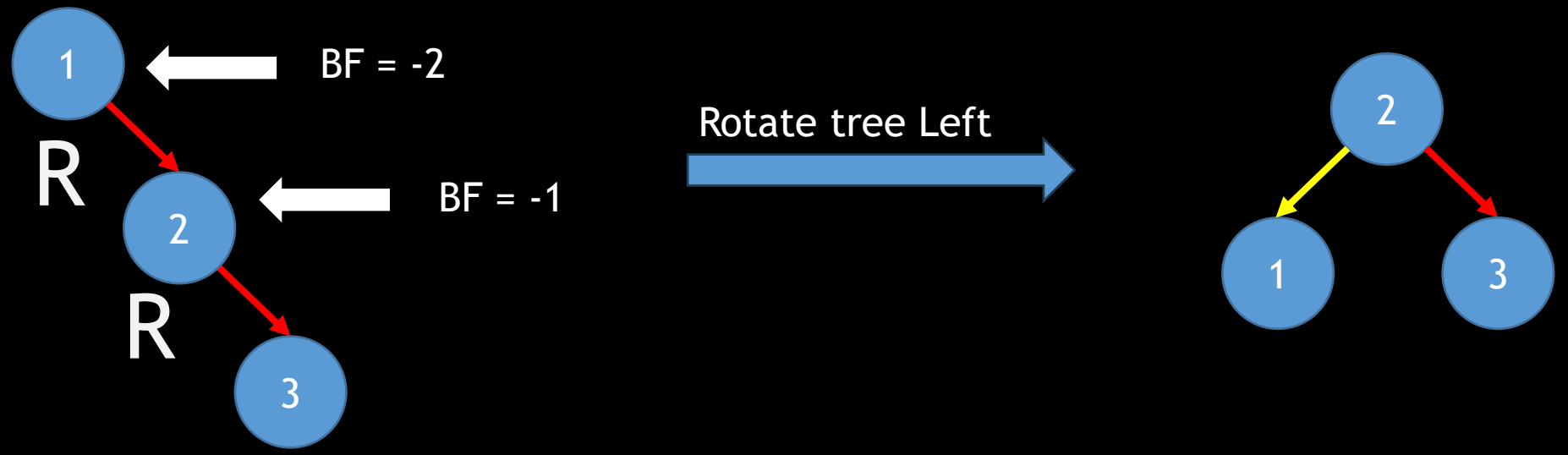
Rotations

- BF(node) > 1, BF(left) >= 0
- Left Left -> Right



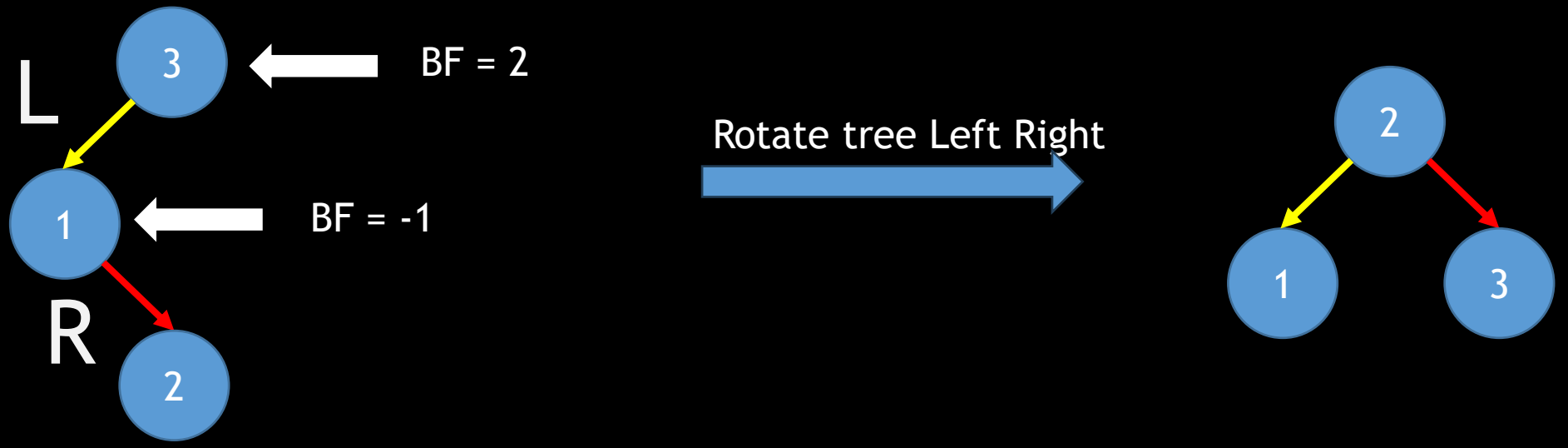
Rotations

- BF(node) < -1, BF(right) <= 0
- Right Right -> Left



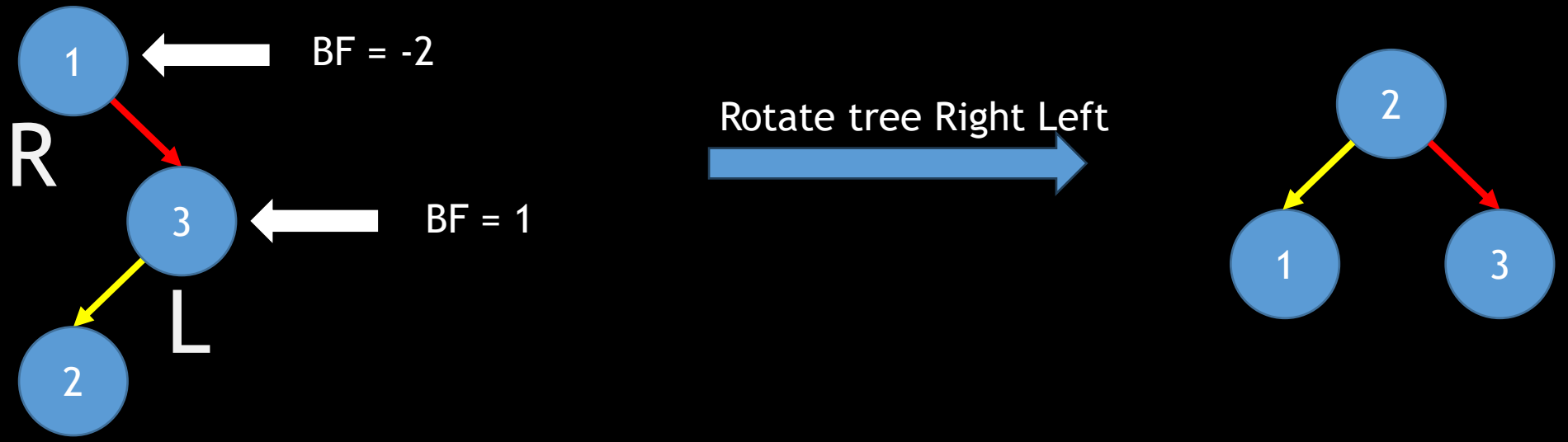
Rotations

- BF(node) > 1, BF(left) < 0
- Left Right -> Left Right



Rotations

- BF(node) < -1, BF(right) > 0
- Right Left -> Right Left



Insertion

```
insertNode (rootNode, newNode)
        if not rootNode
                return newNode
        if rootNode.key == newNode.key
                return rootNode
        if newNode.key < rootNode.key
                rootNode.left = insertNode(rootNode.left, newNode)
        else
                rootNode.right = insertNode(rootNode.right, newNode)
        rootBalanceFactor = findBalanceFactor (rootNode)
        leftBalanceFactor = findBalanceFactor (rootNode.left)
        rightBalanceFactor – findBalanceFactor (rootNode.right)
        if rootBalanceFactor > 1 && leftBalanceFactor >= 0
                return rightRotate (rootNode)
        if rootBalanceFactor < -1 && rightBalanceFactor <= 0
                return leftRotate(rootNode)
        if rootBalanceFactor > 1 && leftBalanceFactor < 0
                return leftRightRotate(rootNode)
        if rootBalanceFactor < -1 && rightBalanceFactor > 0:
                return rightLeftRotate(rootNode)

        return rootNode
```

Deletion

- More complicated than insertion
  - More expensive!
- Starts as per a normal BST deletion
  - Remove the leftmost child of the delete node and keep a reference to it
- Track back up the tree to the actual node to be deleted
  - Rebalance as required
- Replace the actual node to be deleted with the leftmost child that was removed
- Track up to the root
  - Rebalance as required

AVL Trees – Best Uses

- Advantages
  - Always height balanced
  - Max height is $\log_2 n$
  - Better search time than a standard BST
  - Self-balancing

- Retrieval is fast!
- Insertion is less expensive than deletion
- Best used when search is more common & insert/delete less common

- Need to know the problem to choose the best tool for the job

Summary

- AVL trees are an elegant hack
- Based on balancing the tree to prevent degeneracy
- Improve search efficiency at the cost of insertion efficiency